

Storage Classes in C

Storage class is used to describe following points properly of the variable.

1. Location $\rightarrow$ (memory)
2. Default Initial value $\{$ It might be zero or garbage $\}$
3. life $\rightarrow$ $\{$ Existence of the variable $\}$
4. Scope (Range of the variable)
 - Block
 - function
 - Program

```
#include <stdio.h>
```

```
int a = 50;
```

```
int b = 100;
```

```
int main ( )
```

```
{
```

```
printf("%d", b)
```

```
printf();
```

```
}
```

```
{
```

```
int a = 20;
```

```
printf("Inside block %d", a);
```

```
}
```

```
{
```

```
printf("%d", a++);
```

```
}
```

```
{
```

```
void f();
```

```
{
```

```
int a = 30
```

```
printf("%d", a);
```

```
}
```

} Program scope

} Block 1

} Block 2

Here no a is defined inside the block so we will take global a

} function scope

Types of Storage Class :

1. Automatic storage class
2. External storage class
3. Static storage class
4. Register storage class

• Auto

Syntax: `auto datatype variable name ;`
or

`datatype variable name ;` & It is not necessary to write `auto` ?

```
#include <stdio.h>
```

```
auto int x = 30; X
```

```
int main ( )
```

```
{
```

```
    auto int x = 10;
```

```
    {
```

```
        auto int x;
```

```
        printf("%d", x);
```

```
    }
```

```
    printf("%d", x)
```

```
}
```

Default Initial value → garbage value

Scope → Block / function

Lifetime → within the block

location → stack

• Register

syntax: `register int a;`

Default initial value → garbage value

Scope → Block/function

lifetime → within the block

location → CPU register

Register keyword hints the compiler to store a variable in register memory.

This is done because access time reduces greatly for most frequently referred variables.

It depends on many condition whether the compiler puts the given variable in register or not.

If the variable declared as register and doesn't get space in CPU register then they act as auto.

We can't get the address of a register variable with the help of a pointer. ? Because the variable is not having any memory location it is storing in register ?

• Static

Syntax: `static int x;`

Default value → 0

scope → Block / function

lifetime → till the end of the program

location →

```
#include <stdio.h>
```

```
void display();
```

```
int main()
```

```
{
```

```
    display();
```

```
    display();
```

```
    print("x = %d", x);
```

```
}
```

```
void display()
```

```
{
```

```
    static int x = 0;
```

```
    x += 10;
```

```
    printf("In x = %d", x);
```

```
}
```

X Because static variable have scope within the block

• Static variables have the property of preserving their value even after they are out of their scope.

Output ÷

10

20

static variable get initialize only once then after it retains its value.

• Extern

syntax: `extern datatype variable_name;`

Default initial value $\rightarrow 0$

scope $\rightarrow$ Program

lifetime $\rightarrow$ till the end of the program

location $\rightarrow$

Extern is used when a particular file needs to access a variable from another file.

`int var;`

`extern int var;`

$\downarrow$
we need to only declare it because here we are simply accessing the variable which is defined in any other file, or block.


```
#include <stdio.h>
```

```
void fun1();
```

```
void fun2();
```

```
int main()
```

§ But after writing this line it will access a
// extern int a; from where it is defined

printf("%d", a) X It will give error because a is
fun1(); not declared.

```
fun2();
```

```
§
```

```
void fun1()
```

```
§
```

```
int a = 2; a++;
```

```
printf("a = %d", a);
```

```
§
```

```
void fun2()
```

```
§
```

```
printf("Hello from fun2");
```

```
§
```

```
int a;
```

file.c

```
#include <stdio.h>
#include "support.c"
int x = 10;
extern void display();
void main ()
{
    display ();
}
```

support.c

```
#include <stdio.h>
void display()
{
    extern int x;
    x++;
    printf("Hello from support file");
    printf(" x = %d", x);
}
```